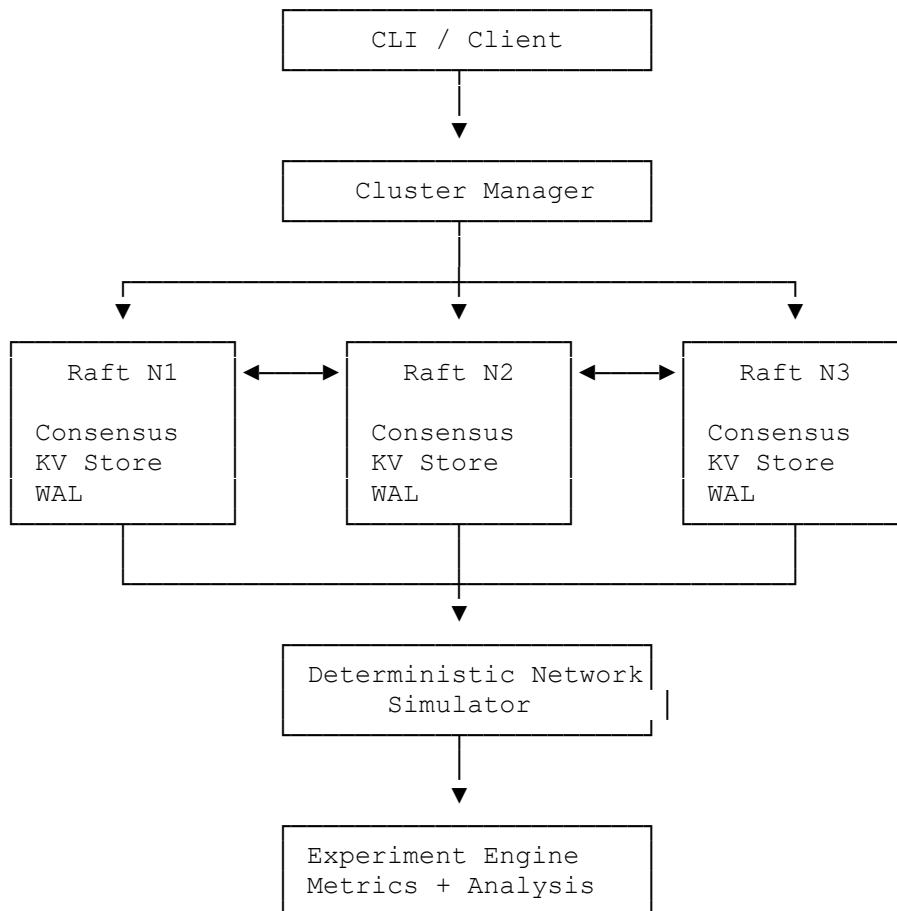


PyRaft Lab 2.0 — Upgraded Version

1. Upgrade the Architecture

Keep the existing architecture:



The original already separates Raft, KV, networking, client, experiments and CLI.

The upgrade is to add **observability and cluster management around those components**.

2. Add a Cluster Controller

Create:

```
src/pyraft_lab/cluster/
```

with:

```
ClusterManager
NodeManager
```

ClusterConfig
LifecycleManager

It should handle:

```
pyraft-lab cluster start --nodes 5
pyraft-lab cluster stop
pyraft-lab cluster restart node-2
pyraft-lab cluster status
pyraft-lab cluster topology
```

This makes the project feel much more like an actual distributed-system tool.

3. Add Real-Time Cluster Observability

This is the **biggest upgrade I recommend**.

Your current project generates JSON + PNG results.

Add a lightweight local dashboard.

Not a giant web application.

Something like:

PyRaft Lab Dashboard

Cluster: 5 Nodes	Term: 17
Leader: node-3	Status: HEALTHY

N1	N2	N3	N4	N5
Follower	Follower	LEADER	Follower	Follower

Commit Index: 1842
Applied: 1842
Log Entries: 1850

Election Time: 182 ms
P99 Latency: 41 ms
Availability: 99.97%

Then show:

- leadership timeline
- commit index
- log length
- request rate
- latency
- dropped packets

- elections
- node health

This will make your demo **far more impressive**.

4. Add an Event/Trace System

Create:

```
src/pyraft_lab/observability/
```

Every important event gets recorded:

```
ELECTION_STARTED  
VOTE_REQUESTED  
VOTE_GRANTED  
LEADER_ELECTED  
APPEND_ENTRIES_SENT  
APPEND_ENTRIES_REJECTED  
LOG_COMMITTED  
NODE_CRASHED  
NODE_RECOVERED  
PARTITION_CREATED  
PARTITION_HEALED  
TERM_CHANGED
```

Example:

```
{  
  "timestamp": 4210,  
  "node": "node-3",  
  "term": 8,  
  "event": "LEADER_ELECTED",  
  "details": {  
    "votes": 3,  
    "cluster_size": 5  
  }  
}
```

Now you can reconstruct **exactly why something happened**.

This is extremely valuable when debugging distributed systems.

5. Add Deterministic Replay

You already have a virtual clock and deterministic network simulator.

Take this further.

Every experiment should generate:

```
run_id
random_seed
scenario
network configuration
fault timeline
workload
event trace
```

Then:

```
pyraft-lab replay --run run_2026_001
```

should reproduce the same failure.

This is a **fantastic distributed-systems feature**.

You can say on your resume:

Built deterministic fault injection and replay infrastructure for reproducible distributed-system failures.

That's much stronger than simply:

Built a Raft implementation.

6. Add Jepsen-Style History Validation

Your current specification already requires recording concurrent request histories and checking linearizability.

Upgrade it into:

```
Client Operations
  ↓
History Recorder
  ↓
Operation Timeline
  ↓
Linearizability Checker
  ↓
PASS / FAIL
  ↓
Failure Explanation
```

Instead of simply:

```
Linearizability: FAIL
```

produce:

Linearizability: FAIL

Violation:

PUT(x, 10)

GET(x) → 5

Node: N4

Term: 12

Time: 14.382s

Possible cause:

Stale read from partitioned leader.

That would be an excellent feature.

7. Add Automatic Fault Campaigns

Currently you have seven predefined scenarios.

Keep those.

But add:

```
pyraft-lab stress --nodes 5 --duration 60s
```

The system automatically generates combinations of:

leader crash

+

packet loss

+

latency

+

partition

+

node restart

For example:

Trial 001

Seed: 48291

0s normal

10s leader crash

14s leader recovered

20s 20% packet loss

31s network partition

36s partition healed

45s node restart

Run:

100 trials

and produce:

stress-report.json

with:

```
Trials:           100
Passed:           97
Failed:           3
Data loss:        0
Linearizability:  98
Mean recovery:    421ms
P99 recovery:     891ms
```

This is a major portfolio upgrade.

8. Add Fault-Injection DSL

Instead of only YAML:

```
faults:
  - type: node_crash
    target: node-1
```

allow chained experiments:

```
experiment:
  name: cascading_failure

  faults:
    - at: 10s
      action: crash
      target: leader

    - at: 15s
      action: packet_loss
      rate: 0.20

    - at: 20s
      action: partition
      groups:
        - [node-1, node-2]
        - [node-3, node-4, node-5]

    - at: 30s
      action: heal_partition
```

This makes the experiment engine much more flexible.

9. Upgrade Persistence

Your current design uses WAL with `aiosqlite` or append-only files and requires `fsync` before responding to important RPCs.

Add:

```
WAL
↓
Checksum
↓
Crash Recovery
↓
Log Validation
```

Every WAL record can contain:

```
term
index
command
checksum
```

On restart:

```
Loading WAL...
Validating records...
Recovering state...
Replaying committed entries...
Recovery complete.
```

```
Recovered:
Term: 18
Entries: 12,421
Commit Index: 12,398
```

Then test **corrupted WAL recovery**.

That's an excellent edge case.

10. Add Snapshots as a Real Feature

The original specification puts snapshots as a deferred item.

I would upgrade this from:

future work

to:

implemented feature

Flow:

```
Log grows
↓
Threshold reached
↓
Create KV snapshot
↓
Persist snapshot
↓
Compact WAL
↓
Continue Raft
```

Then add:

```
pyraft-lab snapshot --node node-3
pyraft-lab inspect-snapshot --node node-3
```

You don't need network snapshot transfer if time is limited.

11. Add Performance Benchmarking

Create:

```
pyraft-lab benchmark
```

Measure:

```
Throughput
Latency
P50
P95
P99
Election time
Recovery time
CPU
Memory
```

Compare:

```
3 nodes
5 nodes
7 nodes
```

and:

```
0% packet loss
5%
10%
20%
```


Generate graphs automatically.

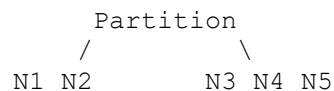
Your existing result schema already includes latency, leadership and fault-impact metrics, so this is a natural extension rather than a completely new subsystem.

12. Add a "Consistency vs Availability" Experiment

This would be **excellent for your project presentation**.

Experiment:

5-node cluster



Show:

Minority side:
Writes → rejected

Majority side:
Writes → accepted

Then heal:

```
Partition healed
↓
Log reconciliation
↓
Followers catch up
↓
Consistent state
```

This directly demonstrates why consensus algorithms work.

13. Add Chaos Testing

Don't make this another huge framework.

Create:

```
pyraft-lab chaos --nodes 5 --duration 120
```

Randomly perform:

```
kill leader
restart follower
drop packets
delay packets
partition network
heal network
increase latency
generate workload
```

The important part:

The system should never silently lose committed data.

At the end:

CHAOS TEST REPORT

```
Trials: 50
Data-loss violations: 0
Safety violations: 0
Linearizability: 49/50
Leader elections: 183
Avg recovery: 384ms
```

14. Upgrade the CLI

Instead of only:

```
run
show-log
status
compare
serve
```

which your original spec defines,

I'd make:

```
pyraft-lab init
pyraft-lab cluster start
pyraft-lab cluster status
```

```
pyraft-lab run
pyraft-lab stress
pyraft-lab chaos
```

```
pyraft-lab node inspect
pyraft-lab node logs
```

```
pyraft-lab trace
pyraft-lab replay
```

```
pyraft-lab benchmark
pyraft-lab compare
```

```
pyraft-lab snapshot
```

Now it feels like an actual developer tool.

15. Add Architecture Quality

Organize it like:

```
pyraft_lab/
├── raft/
│   ├── node.py
│   ├── state.py
│   ├── election.py
│   ├── replication.py
│   ├── persistence.py
│   └── snapshot.py
├── kv/
│   ├── store.py
│   └── commands.py
├── network/
│   ├── simulator.py
│   ├── channel.py
│   ├── faults.py
│   └── clock.py
├── cluster/
│   ├── manager.py
│   ├── config.py
│   └── lifecycle.py
├── client/
│   ├── client.py
│   └── workload.py
├── experiments/
│   ├── runner.py
│   ├── stress.py
│   └── chaos.py
├── observability/
│   ├── events.py
│   ├── tracer.py
│   └── metrics.py
├── consistency/
│   ├── history.py
│   └── linearizability.py
└── dashboard/
```

└─ cli/

This is much more professional.

16. What NOT to Add

This is important.

Your original specification explicitly keeps several things out of scope:

- multi-Raft sharding
- TLS/authentication
- real network deployment
- Byzantine fault tolerance
- cluster membership changes
- web UI as a core requirement

Don't suddenly try to implement all of them.

They will destroy your timeline.

For a 15-day student project, I'd prioritize:

● **Must Have**

1. Raft consensus
2. KV store
3. WAL
4. Network simulator
5. 7 fault scenarios
6. Linearizable reads
7. Unit/integration tests
8. CLI
9. Metrics
10. Deterministic replay

● **Strong Upgrade**

11. Snapshots
12. Event tracing
13. Stress testing
14. Chaos testing
15. Benchmarking
16. Cluster controller

● **If Time Remains**

- 17. Local dashboard
- 18. Advanced visualization
- 19. Snapshot inspection
- 20. More workload types